

ABSTRACT

We developed two machine learning algorithms to classify MRI brain scans images. The first algorithm is a Random Forest Classifier which uses 10 PCA project features for binary classification through tumour detection. The Random Forest achieves an accuracy of 83% when tested on unseen test data. The second algorithm is a convolutional neural network (CNN) classifier which uses reduced images with a size of 32x32x3 for multiclass classification through tumour type identification. The CNN achieves an accuracy of 82% on unseen test data.

Index Terms— Medical Images Classification – Machine Learning – Deep Learning – Random Forest – CNN

1. INTRODUCTION

In this project, we wish to develop two machine learning algorithms to detect and identify the type of brain tumours in MRI brain scans. Machine learning algorithms are computer programs that can be trained using past-data (e.g., MRI scans of patients with and without tumours) to perform complex tasks (e.g., identifying tumours in an MRI scans) with great accuracy.

To tackle this challenge, we had access to the Brain Tumour Classification (MRI) labelled dataset. Using this dataset, we implemented supervised learning techniques, which use labelled examples to infer a function describing the relationship between the inputs and the outputs.

In fact, the type of machine learning algorithm to use for a task depend on the nature of the problem we need to solve. Detecting tumours and identifying their type (among a discrete number of types) are both classification problems. This type of problem concerns tasks where the machine learning algorithm needs to output a discrete class (e.g., “Tumour” or “No Tumour”) based on a number of inputs. Hence we implemented classification algorithms to tackle our problems.

Different classifiers can be built to classify tumours using an image datasets. The most performant algorithm can be determined by trying different classifiers on the dataset provided. Moreover, every classifier has different hyperparameters that can be varied to improve its performance. Hence, we tested many different classifiers, varying their hyperparameters along the way and analysing their performance, to select the most performant one.

In this report, we start by a literature review of some classical and state-of-the-art machine learning algorithms. Then, we describe the two most performant models we selected to tackle our two classification tasks. Afterward, we provide details of the practical implementation of the two models. Then, we present and analyse the experimental

results we obtained using the two models selected. Finally, we present a summary of the findings and discuss directions for future improvements.

2. LITERATURE SURVEY

In this section, we give an overview of some classical and state-of-the-art machine learning algorithms that have been considered for our classification tasks.

In [1], two machine learning classifiers are compared: K-Nearest-Neighbors (KNN) and Support Vector Machine (SVM). In a KNN, for every input data point X , the algorithm identifies the K number of inputs that are closest (using the Euclidian distance for example) to X and assumes that they belong to the same output class as X . On the other hand, an SVM tries to maximise the margin between data that belong to different classes to more easily identify boundary decision separations for different classes. In [1], both the KNN and SVM were tested to classify images based on four classes: airplanes, cars, faces, and motorbikes. It is shown that the SVM outperforms the KNN with an accuracy of 92% for the SVM against 78% for the KNN. However, as stated in [1], there is no classification technique that works best on any given problem. Hence, to determine the most performant algorithm to classify a dataset a multiple techniques have to be tried out and the results have to be compared rigorously.

Another performant classifier is the Random Forest. This is a type of ensemble technique that combines multiple decision tree classifiers based on the different features available. In [2], improved Random Forests classifiers using weighted features and non-valuable trees removal are tested on four different images dataset. When comparing the novel approach to a classic Random Forest algorithm (developed by Breiman) the improved Random Forest method demonstrates a mean accuracy of 88% (for the best number of features selected) across the four datasets. However, the classic Random Forest classifier still performs very well with a mean accuracy of 86%.

Another type of machine learning method called deep learning, has proven to be incredibly performant for a lot of classification tasks. Deep neural networks (DNNs) try to replicate the structure and operation of the human brain by their organisation: a succession of neurons that communicate between each other to infer and transmit information. Today, convolutional neural networks (CNN) are considered the state of the art algorithms for image classification. In [3], a CNN is used for lung image patch classification. Using only one convolutional input layer with kernel size of 7×7 pixels and four hidden layers, the CNN outperforms all the other classifiers compared, with a precision of 74% precision and a recall of 77%.

3. DESCRIPTION OF MODELS

To identify the most performant classifier for each task, we first tested multiple models and dimensionality reduction (DR) methods. For each model and DR method, we identified the best main hyperparameter by varying it and comparing the classification performance. We compared the models using the following performance metrics (considering that “no_tumor” is negative and “tumor” is positive):

- Accuracy: The percentage of correct predictions. It is the most used metric to quantify the performance of a classifier.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

- Precision: The proportion of correct positive predictions (true positives) over the total number of positives predictions. This is a very useful metric when the two classes are un-equally distributed in a dataset [4].

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- Recall: The proportion of correct negatives predictions over the total number of positives values.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

For both models, we tested the two following DR techniques:

- Analysis of Variance (ANOVA), a supervised DR technique
- Principal Component Analysis (PCA), an unsupervised DR technique

To identify the best classifier, we separated the initial 3000 samples dataset into training and validation (80%/20% split) and analysed the performance on the validation set predictions.

3.1. Task 1: Binary Classification

For the first task, we tested six classifiers: KNN, Decision Tree, Random Forest, KNN with Bagging Ensemble method, Decision Tree with Boosting Ensemble method, and SVM. The tests results for every classifier can be checked in Table A1 in the Appendix.

Using the results presented in Table A1, and the confusions matrix of every model, we decided to use a **Random Forest using 10 PCA** projected features for Task 1. This classifier has the most reliable performance scores with an **accuracy of 94%, a precision of 95%, and a recall score of 99%** on the validation data.

A Random forest is an ensemble methods which has been developed to avoid overfitting¹ in the Decision Tree classifier. Random Forests build a number of decision trees based on the different features available as it can be seen in figure 1 below.

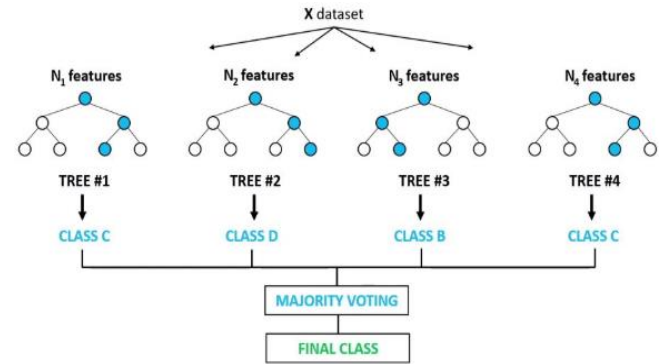


Fig. 1 Random Forest Classifier Organisation [5]

The Random Forest use the bagging technique to create a tree for every different bootstrap² sample of the training dataset. Each decision tree fits a different training sub-set with a specific classification performance. All trees in a Random Forest slightly overfits their training sub-set. This helps reduce the correlation between each trees and between each predictions or prediction errors [6]. After all trees are fitted, predictions from the trees are then averaged. Then, for a classification problem, the random forest prediction for a new sample is the majority vote for the class label across the trees in the ensemble.

For Random Forests, the improvement over regular bagging resides in the fact that each time a tree node is created, a number q of features are chosen randomly and the data is separated using the best feature to split the data. By reducing the features to a random subset that may be considered at each split point, we increase the differences between the prediction and predictions errors of each tree. When the predictions are then averaged, it produce a more accurate final prediction.

We also decided to use PCA to reduce the dimensionality of the feature space of the images. This is an unsupervised feature selection that projects the original set of

¹ Overfitting happens when the model fits too well the training data but will not be able to perform well on unseen new data

² A bootstrap is a sample of the training dataset, where data points may appear more than once in the overall sample, referred to as sampling with replacement. [6]

features onto a smaller one which capture as much variation in the data as possible.

3.2. Task 2: Multi-Class Classification

Since a multi-class classification problem is, in most cases, more challenging than a binary classification task, we assumed that the classifiers tested in Task 1 would not perform better in Task 2. Hence, we tested a Random Forest, a Multi-Layer-Perceptron (MLP), and a CNN to identify the type of tumours in the images of the dataset. The tests results for every classifier can be observed in Table A2 in the Appendix.

After comparing the performance of the algorithms, we have decided to use a CNN for Task 2. The CNN achieves an accuracy of 83% on the validation set using 32x32x3 image data size. While a CNN with higher dimensional images achieves a higher accuracy, the time it takes to train the CNN with larger images is too expensive for us to use.

A CNN is a type of deep neural network (DNN). A DNN is an algorithm composed of layers. The first layer is the input layer, which receive information through different forms depending on the type of network. The last layer is the output layer, which generate the predictions (labels in our case) based on the inputs. The layers in-between are called hidden layers because we do not have access to them after they have been defined. Each layer is composed of a number of neurons. A CNN is composed of convolutional layers, pooling layers, and fully-connected layers, as it can be seen in Figure 2 below.

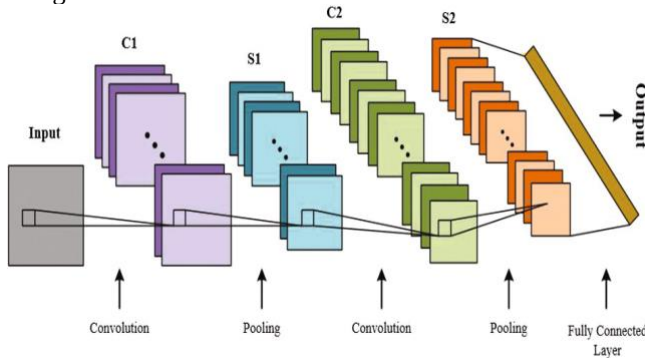


Fig. 2: CNN Architecture [7]

Convolutional layers are filters that automatically learns the internal features of the dataset using squares of input data. By repeatedly applying the same filter to an input sample, convolutional layers produce feature maps, indicating the locations and strength of a detected feature in the input [8]. Convolutional layers constitute the innovation of CNNs, giving them the ability to automatically learn a large number of filters in parallel specific to a training dataset. The result is highly specific features that can be detected anywhere on input images [8]. Pooling layers

consolidate the features learned in the convolutional layers feature map. They are used to reduce overfitting of the data. Finally, fully-connected layers are the normal neural network layers that transmit data. They are used at the output of the CNNs to compute the probability of every output label and thus to make the prediction on the most probable label. CNN are particularly well suited for image classifications tasks because they preserve the spatial structure of images [9]. Indeed, other algorithms we have tested so far treated the inputs as vectors, disregarding the spatial relations between the pixels of the images, but CNNs work extremely well with image matrices as inputs.

4. IMPLEMENTATION

• Dataset Description

The dataset is composed of 3000 512x512x3 pixel gray-scale MRI images (.jpg) organized in 4 classes. A csv file containing their corresponding classes (labels) is also provided. The four possible labels are: no_tumor, meningioma_tumor, glioma_tumor, and pituitary_tumor. We are also provided with 200 additional images for testing. In this project, we assumed that the dataset does not contain erroneous or null sample. Figures 3.a and 3.b below present two example of images in the dataset.

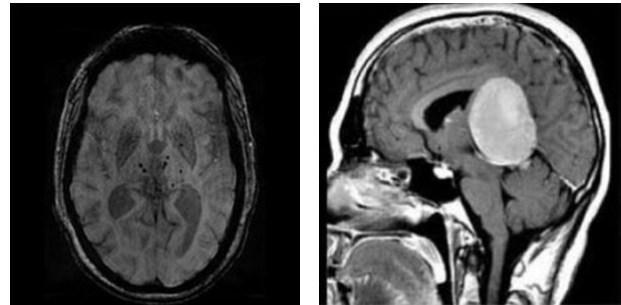


Fig. 3.a and 3.b: MRI brain scan without a tumour and with a meningioma tumour respectively

4.1. Task 1: Binary Classification

• Data Loading and Processing

For Task 1, the *dt.process_data()* is used to do most of the data processing in three steps, using function in our *data_processing* package:

1. It calls the *load_image()* method which opens every image file using the *imageio* library and saves it in the *images_array* numpy array with final size 3000 (number of samples) x 512x512 (number of pixels) x 3 (RGB values in each pixel).
2. It calls the *image_array_to_vector()* method to flatten every image which is stored as a 512x512x3 array. As a result, every image is now stored as a 512x512x3 =

786432 pixel vector in the 3000x786432 numpy array called *image_vectors*

- It calls the *load_labels()* method which opens the csv file containing the labels. A 0 is stored in the *labels* numpy array if no_tumor is read, a 1 is stored otherwise.

• Dimensionality Reduction

We have selected the PCA DR technique with 10 features projected.

The PCA and ANOVA features (5 and 10) have already been computed for both tasks using the *extract_feature* program in the Function package. The program calls the *dt.process_PCA_features()* method located in the data processing file. The method saves the features in a csv file called 'Features' in Tasks 1 and 2 folders. Hence, to get the PCA features in the final model program, we just have to call the *get_features()* method, specifying the path to the features file.

• Cross-Validation and Dataset-split

In order to evaluate the performance of our model, while changing the hyperparameters, we need to split the dataset into training and validation sub-sets. The training set will be used by the model to learn the function to classify the data, and the validation set will be used afterwards to test the performance of the model for each hyperparameters configuration.

We decided to leave the 200 sample testing dataset provided only for the final analysis of our model performance. The initial 3000 samples dataset is split between 80% training and 20% validation. However, by partitioning the available data, results can be dependent of the particular split that we perform. To avoid this issue, we first used non-exhaustive k-fold cross validation method. This technique divides all the samples in k groups (or folds) of samples of equal sizes. The model is trained using k-1 folds, and the fold left out is used for validation. We thus perform k-fold cross validation using 5 iterations using the *cross_validate()* method of the sklearn library. We compared the standard deviation across the 5 splits for accuracy, precision and recall.

We confirmed that the different split for the dataset does not alter radically the performance of the model, with the standard deviation across the 5 splits being less than 1 for accuracy, precision and recall. Hence, using the *train_test_split()* method from the sklearn library, we separated the dataset into training (80%) and validation (20%) sub-set.

• Hyper-parameters tuning

In order to maximise the performance of our Random Forest classifier, we tuned the hyperparameters as described in Table 2 below:

Hyperparameter	Description	Values Tested	Number of Values Tested
<i>criterion</i>	Measures the quality of a split	gini/entropy	2
<i>n_estimators</i>	The Number of decision trees in the forest	range(20, 200,20)	9
<i>max_features</i>	The Number of random features to consider at each split	[1,2,3,4,5]	5
<i>max_depth</i>	Maximum depth of the decision trees	[1,2,3,4,5,6, 7,8,9,10]	10
<i>min_samples_split</i>	The minimum number of samples required to split an internal node	[2,3,4,5]	4
Total Number of Hyperparameters Configurations Tested			3600

Table. 2: Hyperparameters tuning for Task 1

We performed an exhaustive search over the hyperparameters values using the *GridSearchCV()* function of the sklearn library to maximise the accuracy. The accuracies, precisions and recall scores for every configurations were computed, sorted (ranked by accuracy score), stored in a pandas Data Frame, and saved in a csv files called *Tuning_Results_by_Accuracy.csv*.

The best model hyperparameters are presented in Table 3 below.

Hyperparameter	To Maximise Accuracy	Scores
<i>criterion</i>	entropy	Accuracy = 93%
<i>estimators</i>	140	Precision = 93%
<i>max_features</i>	5	Recall = 99%
<i>max_depth</i>	10	
<i>min_samples_split</i>	2	

Table. 3: Best Random Forest Hyperparameters for Accuracy Maximisation

• Final Model Training with best hyperparameters

Finally, we create a new *RandomForestClassifier()* object (sklearn library) with the best hyperparameters computed and train it with the training set. We then plot the learning loss and validation accuracy curves of the model using the method *plot_learning_curve()* of the data_processing package which was modified from the one proposed in [10], and uses the *learning_curve()* function of the sklearn library. Then, we plot the confusion matrix for the

validation set using the *ConfusionMatrixDisplay()* object of the sklearn library.

- **Final Model Testing with best hyperparameters**

Finally, we process the 200 samples dataset into features and labels vectors and use the trained classifier to make the final predictions. We compute the final testing accuracy, precision, and recall scores using the sklearn library. They are saved in the *Final_Test_Results.csv* file. Lastly, the learning loss and validation accuracy curves, and the confusion matrix are plotted.

4.2. Task 2: Multi-class Classification

- **Data Loading and Processing**

For Task 2, the *dt. process_multiclass_data()* is used:

1. It calls the *load_DNN_images()* method which is provided with the argument *gray_2D=False* and *resize=image_size*. Such as before, the method opens every image file using the *imageio* library and save it in the *images_array* numpy array. However, since *resize* is set to (32,32), each image in the dataset is resized from 512x512x3 to 32x32x3.
2. It calls the *image_array_to_vector()* method such as in Task 1, but the images are not processed as vectors by the CNN.
3. It calls the *load_multiclass_labels()* method which opens the csv file containing the labels. It first encodes the labels as numerical labels using the sklearn *LabelEncoder* object. Then, since *one_hot* is set to True, *one_hot* encoding³ is used to store the labels.

- **Dimensionality Reduction**

As mentioned previously, the only dimensionality reduction technique that we will use for the CNN, is resizing the picture as desired to increase computing speed while keeping a relatively high accuracy. This is performed in the data processing part.

- **Baseline Model Definition**

We define the CNN by defining its layers architecture using the TensorFlow and Keras libraries, and particularly the *KerasClassifier()* wrapper. The baseline architecture used in this project is the one used in [11]. It uses

- An input convolutional layer with a relu activation function⁴ with 32 neurons;
- 2 Convolutional/Pooling pairs of hidden layers with a relu activation function with 64 neurons
- A final fully connected hidden layer with 128 neurons;

³ In one-hot encoding each class is converted into a vector of bits where only one bit is 1 and all the other bits are 0.

- An output layer with a sigmoid activation function with 4 neurons.

The model uses the SGD optimiser.

- **Cross-Validation and Dataset-split**

Such as for task 1, we perform k-fold cross validation on the dataset with 5 iterations using this time the *cross_val_score()* method of the sklearn library, since we only need the accuracy. We compared the accuracy standard deviation across the 5 splits. We confirmed that the different split for the dataset does not alter radically the performance of the model, with the accuracy standard deviation across the 5 splits being less than 1. Hence, using the *train_test_split()* method from the sklearn library, we separated the dataset into training (80%) and validation (20%) sub-sets.

- **Training Parameters tuning**

In order to maximise the performance of the CNN, we had initially planned to tune the training model parameters as described in Table 4 below:

Hyper-parameter	Description	Values Tested	Number of Values Tested
<i>batch_size</i>	The number of samples processed before the model automatically updates its weights	[10, 20, 40, 60, 80]	5
<i>epochs</i>	Number of passes of the entire dataset forward and backward through the neural network	[10, 50, 100]	2
<i>optimiser</i>	Method used to minimize the loss function	['SGD', 'Adam']	2
<i>learn_rate</i>	The learning rate of the CNN	[0.001, 0.01, 0.1, 0.2, 0.3]	5
<i>activation</i>	The activation function for the layers	['softmax', 'relu', 'sigmoid']	3
<i>Neurons</i>	Number of neurons in the hidden layers	[10, 15, 20, 25, 30]	5
Total Number of Parameters Configurations Tested			1500

Table. 4: Parameters Tuning for Task 2

⁴ Defines how the weighted sum inputted into the layers neurons is transformed into the output.

We had planned to perform an exhaustive search over the parameters values using the *GridSearchCV()* function such as for Task 1. However, considering that each training stage of the CNN takes approximately 5 minutes, we do not have the computational facilities, or the time to perform such grid search. Hence, the tuning pipeline is provided in the code, but we did not run it. We manually tried various *batch_size* and *epochs* to come up with the most performant configuration.

• Final Model Testing

Finally, we processed the 200 samples dataset into 32x32 images and labels vector and used the trained CNN to make the final predictions. We computed the final testing accuracy and plotted the learning loss and validation accuracy curves.

5. EXPERIMENTAL RESULTS AND ANALYSIS

5.1. Task 1: Binary Classification

Figure 4 below present the Training Loss and Cross-validation accuracy curves for our Random Forest Classifier.

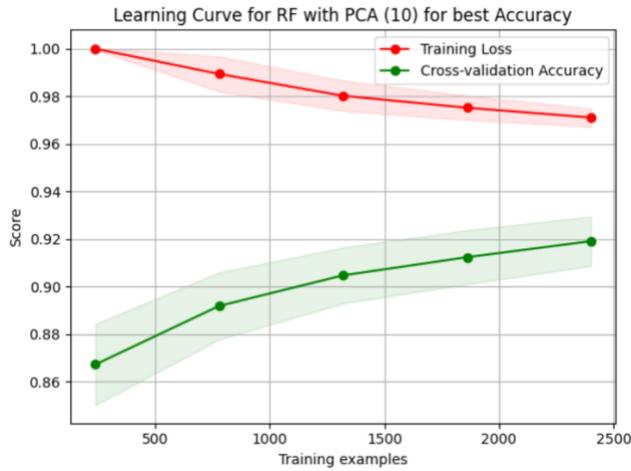


Fig. 4: Random Forest Loss and Accuracy curves

In Figure 4, we can observe two curves: the training loss and the cross-validation accuracy. They represent the evolution of the error during training and the accuracy on the validation; they both are plotted against the increase in the number of samples available to our model for training. The training loss shows a smooth decrease from 1.0 to 0.95, meaning that the model is reducing its error progressively as it is given more data to train with. However, with the accuracy that we are achieving, we would expect that the loss reduced to a lower value, closer to 0. Similarly, the validation accuracy increases almost linearly from 86.5% to 92%, showing a positive correlation between the model's performance when classifying the validation data and the

amount of training sample it uses to train. We can conclude that our model is correctly fitting the training data.

Table 5 below present the accuracy, precision, and recall scores for validation and testing sets using the best hyperparameters for maximising the accuracy.

	To Maximise Accuracy	
	Validation	Testing
Size	600	200
Accuracy	93%	83%
Precision	93%	84%
Recall	99%	98%

Table. 5: Final Performance Scores for Task 1

From Table 5, we can see that we are achieve excellent results on the Validation data. This proves that our model is effectively able to identify if there is a tumour in a MRI brain scan image. While the testing accuracy is lower than the validation accuracy, it is still very high at 83%. Considering the fact that the validation set is three times larger than the testing set, we can hypothesize that the decrease in accuracy is due to the smaller size of the dataset, which increases the importance of the difference between the number of images that have tumours, and the number of images that don't.

We can also observe the validation and testing Confusion Matrices in Figure 5.a and 5.b below.

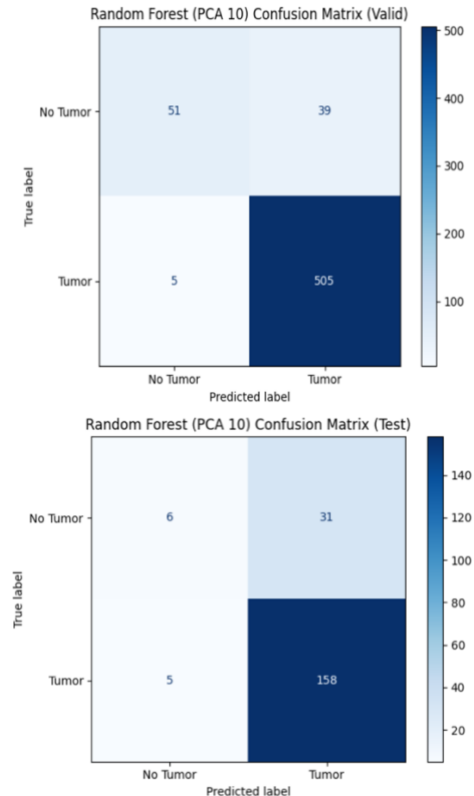


Fig. 5.a and 5.b: Confusion Matrices for Validation and Test Data for Task 1

Finally, in Figure 5.a and 5.b we can observe that the validation and testing confusion matrices are quite similar. The number of images classified as having tumours is very high, since a large proportion of each set indeed has tumours. However, we can see that, in proportion, a lot more images are wrongly classified as having tumours for the test set (15.5%) than for the validation set (6.5%). Again, this might be because the smaller size of the testing set is accentuating the importance of the difference between the number of images that have tumours, and the number of images that don't.

5.2. Task 2: Multi-class Classification

Figures 4.a and 4.b present the cross entropy loss and classification accuracy curves when training the CNN with *batch_size* = 64 and *epochs* = 100. Both curves are plotted vs the training epoch.

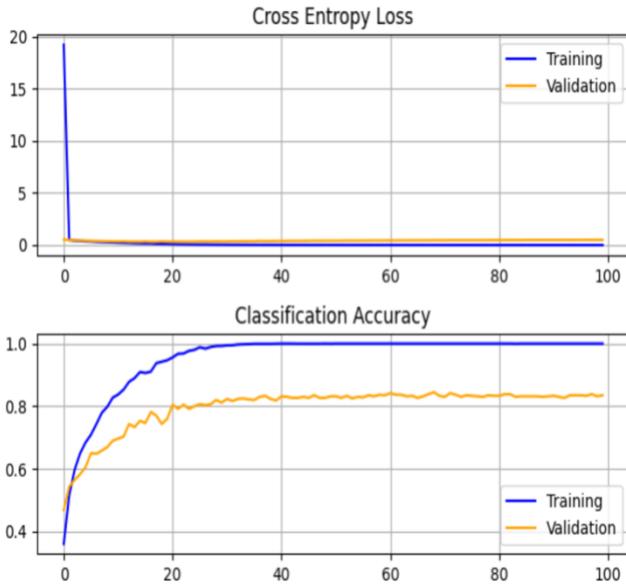


Fig. 6.a and 6.b: CNN Loss and Accuracy curves respectively

In Figure 6.a, we can see that the training loss of the CNN decreases to 1 after the first epoch. In this way, the validation loss is almost 0 from the first epoch. However, after the 20th epoch, while the training has reached 0, the validation loss starts to slightly increase. This indicates that the model starts to overfit the validation set. However, the difference between the training and validation losses is so small that we cannot fully judge if the model is overfitting. More generally, the training and validation loss is almost 0 which demonstrate the performance of our CNN when training with three dimension images, even if we have reduced the dimension to 32*32=1024 pixels.

In Figure 6.b, the classification accuracy for both training and validation increase together almost linearly during the first 5 epochs. During those 5 epochs, the model is effectively learning how to classify the images progressively

and can make prediction on the validation set according to its training accuracy. However, after the 5th epoch, we can clearly observe that the model starts to overfit the training data. This means that the algorithm is learning to classify the specificities of the training data. Hence, after the 5th epoch, the training accuracy continues to grow heavily until reaching 100%, but it is not closely followed by the validation accuracy anymore. However, the validation accuracy still grows to a reach 83%.

Table 6 below present the accuracy scores for validation and testing sets using the best training parameters.

	Validation	Testing
Size	600	200
Accuracy	83%	82%

Table. 6: Final Performance scores for Task 2

From Table 6, we can see that we are achieve great results on both the validation and testing data. Our model is effectively able to identify which type of tumour is present in an MRI brain scan image with an accuracy of 82%.

6. CONCLUSION

In this assignment, our aim was to use machine learning techniques to detect and identify brain tumours in MRI scans of human brains. We built two algorithms using two different classifiers. The first algorithm uses a Random Forest; it is used for binary classification to determine if a tumour is present or not in an MRI brain scan image. The algorithm achieved accuracies of 93% on the validation set and 82% on the test set. The second algorithm uses a CNN for multi-class classification to determine the type of tumour (or no tumour) present in an MRI brain scan image. The CNN algorithm achieved an accuracy of 83% on the validation set and 82% on the test set.

While our results are relatively good, multiple procedures could be implemented in the future to try to improve the performance of our models. For example, We could have used additional data-processing and mining techniques such as median filtering to reduce the noise on the images. Additionally, we could have used other dimensionality reduction techniques such as computing the colour histogram of the images and selecting the most important data points to feed the classifier. Finally, if we had access to a more powerful computer, we could have tune more parameters of the CNN such as the number of layers and the number of neurons in each layer.

7. REFERENCES

- [1] J. Kim, B.-S. Kim and S. Savarese, "Comparing Image Classification Methods: K-Nearest-Neighbor and Support-Vector-Machines," in *6th WSEAS international conference on Computer Engineering and Applications*, 2012.
- [2] X. Baoxun, Y. Yunming and N. Lei, "n improved random forest classifier for image classification," in *IEEE International Conference on Information and Automation*, 2012.
- [3] Q. Li, W. Cai, X. Wang, Y. Zhou, D. Dagan Feng and M. Chen, "Medical image classification with convolutional neural network," in *3th International Conference on Control Automation Robotics & Vision*, 2014.
- [4] J. Jordan, "Evaluating a machine learning model," Jeremy Jordan, 17 July 2017. [Online]. Available: <https://www.jeremyjordan.me/evaluating-a-machine-learning-model/>. [Accessed 15 November 2021].
- [5] D. David, "Random Forest Classifier Tutorial: How to Use Tree-Based Algorithms for Machine Learning," freeCodeCamp, 6 August 2020. [Online]. Available: <https://www.freecodecamp.org/news/how-to-use-the-tree-based-algorithm-for-machine-learning/>. [Accessed 10 December 2021].
- [6] J. Brownlee, "How to Develop a Random Forest Ensemble in Python," Machine Learning Mastery, 20 April 2020. [Online]. Available: <https://machinelearningmastery.com/random-forest-ensemble-in-python/>. [Accessed 05 December 2021].
- [7] X. Ren, H. Guo, S. Li, S. Wang and J. Li, "A Novel Image Classification Method with CNN-XGBoost Model," in *IWDW*, 2017.
- [8] J. Brownlee, "How Do Convolutional Layers Work in Deep Learning Neural Networks?," Machine Learning Matery, 19 April 2019. [Online]. Available: <https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/>. [Accessed 12 12 2021].
- [9] J. Brownlee, "When to Use MLP, CNN, and RNN Neural Networks," Machine Learning Master, 23 July 2018. [Online]. Available: <https://machinelearningmastery.com/when-to-use-mlp-cnn-and-rnn-neural-networks/>. [Accessed 12 December 2021].
- [10] "Plotting Learning Curves," Scikit Learn, [Online]. Available: https://scikit-learn.org/stable/auto_examples/model_selection/plot_learning_curve.html#sphx-glr-auto-examples-model-selection-plot-learning-curve-py. [Accessed 13 December 2021].
- [11] J. Brownlee, "How to Classify Photos of Dogs and Cats (with 97% accuracy)," Machine Learning Mastery, 17 March 2019. [Online]. Available: <https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-to-classify-photos-of-dogs-and-cats/>. [Accessed 12 December 2021].

8. APPENDIX

Classifier	Dimensionality Reduction	Best Hyper-parameter	Accuracy	Precision	Recall	Time Taken (s)
KNN	ANOVA (5)	K = 12	86%	89%	96%	0.04
	ANOVA (10)	K = 12	86%	90%	94%	0.03
	PCA (5)	K = 3	89%	92%	95%	0.03
	PCA (10)	K = 1	93%	96%	97%	0.03
Decision Tree	ANOVA (5)	N/A	86%	88%	97%	0.01
	ANOVA (10)	N/A	88%	93%	92%	0.01
	PCA (5)	N/A	90%	94%	93%	0.05
	PCA (10)	N/A	92%	95%	95%	0.07
Random Forest	ANOVA (5)	N_Trees = 150	88%	91%	95%	0.55
	ANOVA (10)	N_Trees = 11	91%	94%	96%	1.23
	PCA (5)	N_Trees = 210	94%	94%	99%	1.25
	PCA (10)	N_Trees = 30	94%	95%	99%	1.27
KNN Bagging	ANOVA (5)	K = 11	85%	89%	94%	0.05
	ANOVA (10)	K = 9	88%	91%	95%	0.05
	PCA (5)	K = 14	92%	92%	98%	0.09
	PCA (10)	K = 14	92%	92%	98%	0.09
Decision Tree Boosting	ANOVA (5)	Estimators = 15	86%	87%	98%	0.05
	ANOVA (10)	Estimators = 10	85%	92%	91%	0.05
	PCA (5)	Estimators = 5	88%	88%	99%	0.03
	PCA (10)	Estimators = 7	88%	88%	99%	0.03
SVM	ANOVA (5)	Kernel = rbf	85%	86%	99%	0.26
	ANOVA (10)	Kernel = rbf	85%	86%	99%	0.24
	PCA (5)	Kernel = rbf	86%	86%	100%	0.23
	PCA (10)	Kernel = rbf	86%	86%	99%	0.18

Table A1: Classifiers Testing Results for Task 1

Classifier	Dimensionality Reduction	Hyper-parameter	Accuracy	Time Taken (s)
Random Forest	ANOVA (5)	N_Trees = 170	50%	0.45
	ANOVA (10)	N_Trees = 145	53%	0.45
	PCA (5)	N_Trees = 95	76%	0.53
	PCA (10)	N_Trees = 245	80%	1.87
MLP	ANOVA (5)	2 Hidden Layers /sigmoid/adam	28%	0.69
	ANOVA (10)	2 Hidden Layers /sigmoid/adam	28%	0.28
	PCA (5)	2 Hidden Layers /sigmoid/adam	28%	0.48
	PCA (10)	2 Hidden Layers /sigmoid/adam	28%	0.53
	Direct Vector Input	2 Hidden Layers /sigmoid/adam	28%	0.80
CNN	Direct Image input	32x32x3 Images	83%	4.36 minutes
	Direct Image input	100x100x3 Images	85%	37.56 minutes

Table A2: Classifiers Testing Results for Task 2

